

Writing the first JSR-286 portlet

Let us start with the ubiquitous example, writing the *Hello World* portlet. The code is available in liferaybook.googlecode.com.

Before we jump into coding, let's set up the project environment. My preferred build and dependency management environment is Maven. Using this, we will create a new project (use *mvn archetype:generate*). Choose a normal WAR project (option 18 in the menu). For our example, I have given *com.liferaybook.hello-world* and *com.liferaybook* as the groupId, artifactId and the package, respectively. To the generated *pom.xml*, add the following dependencies:

```
<dependency>
<groupId>javax.portlet</groupId>
<artifactId>portlet-api</artifactId>
<version>2.0</version>
<scope>provided</scope>
</dependency>
<dependency>
<groupId>javax.servlet</groupId>
<artifactId>jstl</artifactId>
<version>1.2</version>
</dependency>
```

Now create your first portlet class:

```
package com.liferaybook;

import java.io.IOException;
import javax.portlet.GenericPortlet;
import javax.portlet.PortletException;
import javax.portlet.RenderRequest;
import javax.portlet.RenderResponse;

public class HelloWorldPortlet extends GenericPortlet {

    @Override
    protected void doView(RenderRequest request, RenderResponse response)
        throws PortletException, IOException {
        request.setAttribute("message", "Hello World!");
        getPortletContext().getRequestDispatcher("/index.jsp").
            include(request, response);
    }
}
```

The code should be understandable to any servlet programmer. The similarities with the servlet API are listed in Table 1. Note that these similarities are based on API usage; they cannot correspond one-to-one in the functional or operational level.



Figure 1: Default Liferay page

Similarities of portlet APIs with servlet APIs	
Servlet API	Portlet API
javax.servlet.http.HttpServlet.doGet()	javax.portlet.GenericPortlet.doView()
javax.servlet.ServletException	javax.portlet.PortletException
javax.servlet.http.HttpServlet.Request	javax.portlet.RenderRequest
javax.servlet.http.HttpServlet.Response	javax.portlet.RenderResponse
javax.servlet.RequestDispatcher	javax.portlet.PortletRequestDispatcher

Table 1

Coming back to our code, in the *doView()* method, we first set the request attribute *message* with the String *"Hello World"*, then call *RequestDispatcher* to include the *index.jsp*.

Note, in the Maven structure, you have to place the above code as *src/main/java/com/liferaybook/HelloWorldPortlet.java*.

The *index.jsp* page:

```
<p>${requestScope.message}</p>
```

Using the Unified Expression Language (EL), we print the *message* request attribute set in the *HelloWorldPortlet* class. Note that in the JSPs you need not have the *<html><head><body>* elements defined. This is because your portlet is rendered as a box inside the portal's theme.

The JSPs are placed in the project location: *src/main/webapp/*

A couple of more configurations need to be written before we can deploy our brand new portlet. These are *portlet.xml* and *web.xml*.

The following is what *portlet.xml* looks like:

```
<?xml version="1.0" encoding="UTF-8"?>
<portlet-app
    xmlns="http://java.sun.com/xml/ns/portlet/portlet-app_2_0.xsd"
    version="2.0">
```